Assignment D6-V2: Autonomous Orbital Maintenance Platform – Tool Management and Payload Transport

Which code to check (only one version of each exists): PDDL (Q1) → codes/Q1/Q1_Domain.pddl; PDDL+ (Q2) → codes/Q2/Q2_Domain.pddl. Full verified init/goal/plan output for every instance: codes/Plans/.

Domain & Planning Problem

A single free-climbing maintenance robot retrieves, carries, mounts, uses, and returns tools across storage modules and worksites on an orbital platform. Carrying a tool and having it mounted (usable) are modelled separately, and the robot's carrying capacity is limited. Q1 models this with classical STRIPS+typing PDDL; Q2 extends the same domain with PDDL+ continuous processes and events to capture tool wear, heating, and failure over time.

Modelling Choices

Q1 – Basic PDDL. Types: robot, location, tool, tool-slot, task. A tool is picked up into a free tool-slot (pick-up), mounted into the robot's single mount slot before use (mount-tool, use-tool), and can be unmounted or returned to storage (unmount-tool, return-tool). Carrying capacity is the number of tool-slot objects declared in a problem. Q1_ComplexTask.pddl uses three tools but only two slots, forcing the robot to return a tool mid-route to free capacity — this is the required demonstration that limited carrying capacity shapes the plan.

Q2 – PDDL+. use-tool is split into start-use-tool / stop-use-tool so that wear and heat can accumulate continuously instead of changing state instantaneously. Three numeric fluents were added per tool: wear, temperature, usage-duration. The wear-and-heat process increases all three while a tool is in-use; the cool-down process lowers temperature while idle. The tool-overheats event fires once temperature reaches 100, marking the tool broken (no repair action exists, so this is permanent). stop-use-tool additionally requires ≥5 time units of active use, and start-use-tool refuses to run above 80°, so a tool must genuinely cool before reuse. An in-use-for predicate binds a running session to the specific task that opened it — added after testing exposed that a session started for one task could otherwise be closed out under a different task sharing the same tool.

Plan Output Walkthrough

All six instances were run and verified with ENHSP, including one genuinely unsolvable case (Q2_DurationInfeasible), confirmed as such rather than a search timeout. The only surprise during development was a task-binding bug found while building Q2_DurationFeasible, fixed via the in-use-for predicate (see git history). Full initial-state → goal-state walkthroughs (initial state, goal, and complete verified plan) for every instance, Q1 and Q2 alike, are provided in the Appendix.

Limitations

No repair or tool-replacement action exists, so a broken tool is permanently lost, which is more punishing than a real system would likely allow. move has zero duration, so travel time and climbing-specific risk are not modelled. Multi-robot coordination is out of scope (the assignment specifies a single robot). ENHSP's search is satisficing, not optimal, so plans such as Q2_DurationFeasible's include extra, non-essential waiting rather than the true minimum.

Appendix – Full Init → Goal Walkthroughs

Every instance below was run with ENHSP against its paired domain file (codes/Q1/Q1_Domain.pddl for Q1_*, codes/Q2/Q2_Domain.pddl for Q2_*). Machine-readable copies of the same walkthroughs are also kept in codes/Plans/.

Q1_TrivialTask.pddl

Initial state: robot1 at storageA; storageA ↔ worksite1; wrench at storageA; slot1 and the mount slot free; task tighten-bolt at worksite1 requires wrench.

Goal: (task-done tighten-bolt)

```
0.0: (pick-up robot1 wrench storageA slot1)
1.0: (move robot1 storageA worksite1)
2.0: (mount-tool robot1 wrench slot1)
3.0: (use-tool robot1 wrench tighten-bolt worksite1)
```

Walkthrough: the robot picks up the only tool, walks to the only worksite, mounts it, and uses it — four actions, straight line from init to goal.

Q1_ComplexTask.pddl

Initial state: robot1 at storageA; path storageA–worksite1–worksite2–storageB; wrench and drill at storageA, camera at storageB; only slot1/slot2 (2 slots for 3 tools); tighten-bolt (wrench) and drill-hole (drill) at worksite1, inspect-panel (camera) at worksite2.

Goal: (and (task-done tighten-bolt) (task-done drill-hole) (task-done inspect-panel))

```
0.0: (pick-up robot1 drill storageA slot2)
1.0: (move robot1 storageA worksite1)
2.0: (mount-tool robot1 drill slot2)
3.0: (use-tool robot1 drill drill-hole worksite1)
4.0: (unmount-tool robot1 drill slot1)
5.0: (move robot1 worksite1 worksite2)
6.0: (return-tool robot1 drill worksite2 slot1)
7.0: (move robot1 worksite2 worksite1)
8.0: (move robot1 worksite1 storageA)
9.0: (pick-up robot1 wrench storageA slot1)
10.0: (move robot1 storageA worksite1)
11.0: (move robot1 worksite1 worksite2)
12.0: (move robot1 worksite2 storageB)
13.0: (pick-up robot1 camera storageB slot2)
14.0: (move robot1 storageB worksite2)
15.0: (mount-tool robot1 camera slot2)
16.0: (use-tool robot1 camera inspect-panel worksite2)
17.0: (move robot1 worksite2 worksite1)
18.0: (unmount-tool robot1 camera slot2)
19.0: (mount-tool robot1 wrench slot1)
20.0: (use-tool robot1 wrench tighten-bolt worksite1)
```

Walkthrough: drill-hole is done first (steps 0–3) since the drill is already at hand; the drill is then dropped at worksite2 (step 6) purely to free a slot, the robot backtracks all the way to storageA for the wrench (steps 7–9), detours to storageB for the camera (steps 10–14), finishes inspect-panel (steps 15–16), then finally returns to mount the wrench and finish tighten-bolt (steps 17–20) — the 2-slot cap for 3 tools is what forces this back-and-forth shape.

Q2_SimpleTask.pddl

Initial state: Same layout as Q1_TrivialTask, plus wrench's wear = temperature = usage-duration = 0 (cold tool).

Goal: (task-done tighten-bolt)

```
0: (pick-up robot1 wrench storageA slot1)
0: (move robot1 storageA worksite1)
0: (mount-tool robot1 wrench slot1)
0: (start-use-tool robot1 wrench tighten-bolt worksite1)
0: -----waiting---- [5.0]
5.0: (stop-use-tool robot1 wrench tighten-bolt worksite1)
```

Walkthrough: identical setup to Q1_TrivialTask, but use-tool is now start/stop. The plan reaches goal-adjacent state instantly (t=0) then must let 5 time units of the wear-and-heat process elapse before stop-use-tool's usage-duration ≥ 5 precondition is met and task-done fires at t=5.

Q2_ComplexTask.pddl

Initial state: Same layout as Q1_ComplexTask, plus every tool's wear/temperature/usage-duration = 0.

Goal: (and (task-done tighten-bolt) (task-done drill-hole) (task-done inspect-panel))

```
0: (pick-up robot1 drill storageA slot2)
0: (pick-up robot1 wrench storageA slot1)
0: (move robot1 storageA worksite1)
0: (move robot1 worksite1 worksite2)
0: (mount-tool robot1 drill slot2)
0: (move robot1 worksite2 storageB)
0: (pick-up robot1 camera storageB slot2)
0: (move robot1 storageB worksite2)
0: (move robot1 worksite2 worksite1)
0: (return-tool robot1 camera worksite1 slot2)
0: (start-use-tool robot1 drill drill-hole worksite1)
0: -----waiting---- [5.0]
5.0: (stop-use-tool robot1 drill drill-hole worksite1)
5.0: (unmount-tool robot1 drill slot2)
5.0: (mount-tool robot1 wrench slot1)
5.0: (pick-up robot1 camera worksite1 slot1)
5.0: (move robot1 worksite1 worksite2)
5.0: (return-tool robot1 drill worksite2 slot2)
5.0: -----waiting---- [8.0]
8.0: (move robot1 worksite2 worksite1)
8.0: (start-use-tool robot1 wrench tighten-bolt worksite1)
8.0: -----waiting---- [13.0]
13.0: (stop-use-tool robot1 wrench tighten-bolt worksite1)
13.0: (move robot1 worksite1 worksite2)
13.0: (unmount-tool robot1 wrench slot2)
13.0: (mount-tool robot1 camera slot1)
13.0: (start-use-tool robot1 camera inspect-panel worksite2)
13.0: -----waiting---- [18.0]
18.0: (stop-use-tool robot1 camera inspect-panel worksite2)
```

Walkthrough: both mechanisms combine. Capacity (2 slots, 3 tools) drives the pick-up/return/mount juggling seen throughout; every start-use-tool is followed by a mandatory 5-unit wait before its matching stop-use-tool. Total elapsed time to reach the goal: 18.0.

Q2_DurationFeasible.pddl

Initial state: robot1 at storageA; path storageA–worksite1–worksite2; drill at storageA, already warm: temperature = 70, wear = usage-duration = 0; drill-hole-1 at worksite1 and drill-hole-2 at worksite2, both requiring the drill.

Goal: (and (task-done drill-hole-1) (task-done drill-hole-2))

```
0: (pick-up robot1 drill storageA slot1)
0: (move robot1 storageA worksite1)
0: (mount-tool robot1 drill slot1)
0: -----waiting---- [12.0]
12.0: (move robot1 worksite1 worksite2)
12.0: -----waiting---- [34.0]
34.0: (start-use-tool robot1 drill drill-hole-2 worksite2)
34.0: -----waiting---- [40.0]
40.0: (stop-use-tool robot1 drill drill-hole-2 worksite2)
40.0: (move robot1 worksite2 worksite1)
40.0: -----waiting---- [43.0]
43.0: (start-use-tool robot1 drill drill-hole-1 worksite1)
43.0: -----waiting---- [48.0]
48.0: (stop-use-tool robot1 drill drill-hole-1 worksite1)
```

Walkthrough: the drill starts at 70°, close to the 80° restart ceiling. ENHSP's satisficing search lets extra idle time pass (cool-down process running throughout) before ever starting a use, then completes drill-hole-2 and drill-hole-1 in turn. The key point is structural, not the exact timings chosen: the plan is only valid because time is allowed to pass and cool the tool before each start-use-tool — an immediate second use without waiting would push temperature past the 100° failure threshold.

Q2_DurationInfeasible.pddl

Initial state: robot1 at storageA; storageA ↔ worksite1; drill at storageA, already (broken drill) true, wear = temperature = usage-duration = 0; drill-hole at worksite1 requires the drill.

Goal: (task-done drill-hole)

```
Unsolvable Problem
```

Walkthrough: there is no path from init to goal. start-use-tool requires (not (broken ?t)), the drill is broken from the very first state, and no repair action exists anywhere in the domain, so drill-hole can never become task-done. ENHSP confirms this analytically during preprocessing in well under a second, rather than exhausting a search.